# PYTHON ACTIVITY-BASED LEARNING

**Submitted by**

Name: Sambangi Baby Hasini

Roll Number: 26001012

Branch: AIML

Section: CSM-3

**Subject:** Python Programming

**Academic Year:** 2026–2027

# TABLE OF CONTENTS

1. Progressive Electricity Billing
2. Hospital Billing and Patient Classification
3. Campus Event Financial Planning
4. Retail Billing with Multiple Discount Rules
5. Manufacturing Cost and Profitability Analysis
6. Student Feedback Sentiment Scoring
7. Attendance Recovery Computation
8. Fuel Consumption and Route Comparison
9. Progressive Water Tariff Analysis
10. Password Policy Validation Engine

# CASE STUDY 1

## PROGRESSIVE ELECTRICITY BILLING

### 1. Problem Statement

To develop a Python program to calculate the electricity bill based on the number of units consumed using different unit rates.

### 2. Objectives

- To calculate electricity charges.
- To understand conditional statements.
- To apply basic arithmetic operations.
- To display the total electricity bill.

### 3. Assumptions

- Units consumed are entered by the user.
- Electricity charges are calculated based on slabs.
- The number of units cannot be negative.
- The bill is calculated using fixed rates for each slab.

### 4. Algorithm

1. Start the program.
2. Read the number of units consumed.
3. Check whether the units are valid.
4. Calculate the bill according to the applicable slabs.
5. Display the total bill.
6. Stop the program.

### 5. Flowchart

Start → Input units → Calculate bill using conditions → Display bill → Stop.

### 6. Pseudocode

BEGIN
INPUT units
IF units are invalid

DISPLAY "Invalid units"
ELSE
Calculate electricity bill
DISPLAY total bill
END IF
END

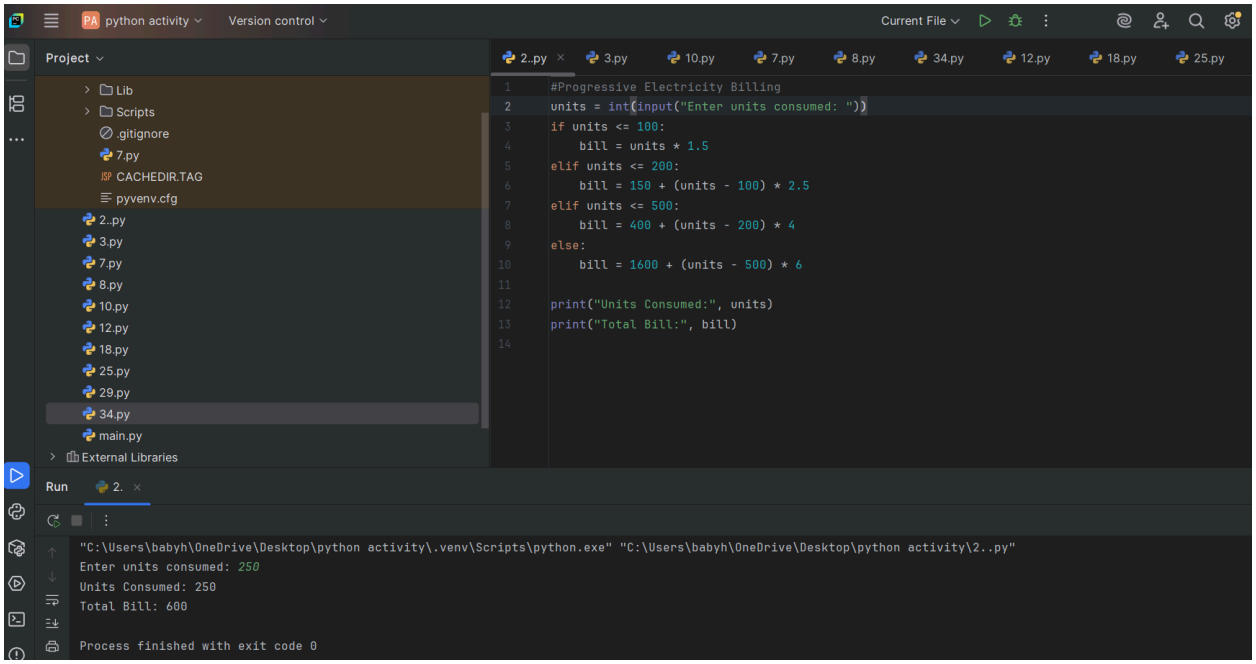
## 7. Python Source Code

(Paste your actual Python program here.)

## 8. Test Cases

| Test | Input | Expected Result |
|---|---|---|
| 1 | 100 units | Bill calculated |
| 2 | 250 units | Bill calculated |
| 3 | 0 units | Zero bill |

## 9. Output Screenshot




### 10. Conclusion

This program helps to calculate electricity bills using different unit rates. It also helps in understanding conditional statements and arithmetic operations.

# CASE STUDY 2

## HOSPITAL BILLING AND PATIENT CLASSIFICATION

### 1. Problem Statement

To develop a Python program to calculate a patient's hospital bill based on the number of days stayed, room type, medicine charges and doctor charges.

### 2. Objectives

- To calculate the total hospital bill.
- To classify patients based on room type.
- To use conditional statements.
- To perform arithmetic operations.

### 3. Assumptions

- Patient details are entered by the user.
- The number of hospital days must be positive.
- Room charges depend on the selected room type.
- Medicine and doctor charges are entered separately.

## 4. Algorithm

1. Start the program.
2. Enter the patient's name.
3. Enter the number of hospital days.
4. Select the room type.
5. Enter medicine and doctor charges.
6. Calculate the total bill.
7. Display the patient's details and total bill.
8. Stop the program.

## 5. Flowchart

Start → Enter patient details → Select room type → Calculate room charges → Add medicine and doctor charges → Display total bill → Stop.

## 6. Pseudocode

```
BEGIN
INPUT patient name
INPUT number of days
INPUT room choice
INPUT medicine charges
INPUT doctor charges
Calculate total bill
DISPLAY patient details
DISPLAY total bill
END
```

## 7. Python Source Code

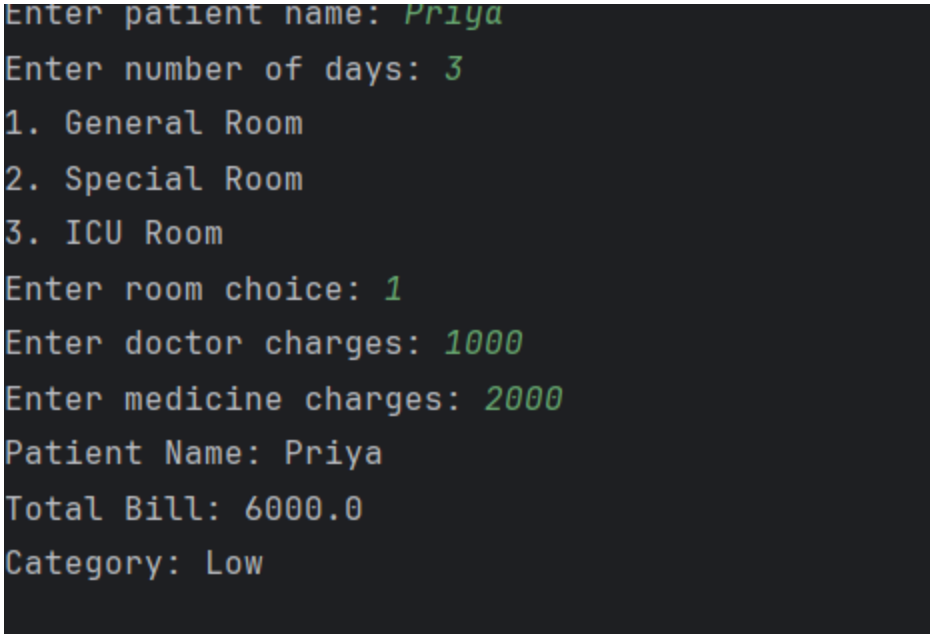
(Paste your actual Python program here.)

## 8. Test Cases

| Test | Input | Expected Result |
|---|---|---|
| 1 | 3 days, General Room | Bill calculated |
| 2 | 2 days, Special Room | Bill calculated |
| 3 | 1 day, ICU Room | Bill calculated |

## 9. Output Screenshot

```python
#Hospital Billing and Patient Classification
name = input("Enter patient name: ")
days = int(input("Enter number of days: "))
print("1. General Room")
print("2. Special Room")
print("3. ICU Room")
choice = int(input("Enter room choice: "))
if choice == 1:
    room_bill = days * 1000
elif choice == 2:
    room_bill = days * 2000
elif choice == 3:
    room_bill = days * 5000
else:
    room_bill = 0
doctor = float(input("Enter doctor charges: "))
medicine = float(input("Enter medicine charges: "))
total = room_bill + doctor + medicine
print("Patient Name:", name)
print("Total Bill:", total)
if total >= 20000:
    print("Category: High")
elif total >= 10000:
    print("Category: Medium")
else:
    print("Category: Low")
```




### 10. Conclusion

This program calculates hospital bills based on room type and other medical charges. It helps in understanding conditional statements and arithmetic operations.

## Case Study 3: Campus Event Financial Planning

1. Problem Statement

Develop a Python program to calculate the income and expenses of a college event and find whether it results in a profit or loss.

2. Objectives

- Calculate total income from registration fees.
- Calculate total event expenses.
- Find the profit or loss.

3. Assumptions

The number of participants, registration fee, and expenses are entered by the user. Expenses include food, decoration, and other costs.

4. Algorithm

1. Start.

2. Input participants and registration fee.
3. Input food, decoration, and other expenses.
4. Calculate total income and expenses.
5. Find the balance.
6. Display profit, loss, or no profit and no loss.
7. Stop.

5. Flowchart

Start → Input Details → Calculate Income → Calculate Expenses → Find Balance → Check Balance → Display Result → Stop.

6. Pseudocode

START
INPUT participants, registration fee
INPUT food, decoration, other expenses
income = participants × registration fee
expenses = food + decoration + other expenses
balance = income − expenses

IF balance > 0, DISPLAY Profit
ELSE IF balance < 0, DISPLAY Loss
ELSE DISPLAY No Profit No Loss

STOP

7. Python Source Code

```
participants = int(input("Enter number of participants: "))
registration = float(input("Enter registration fee per person: "))
food = float(input("Enter food expenses: "))
decoration = float(input("Enter decoration expenses: "))
other = float(input("Enter other expenses: "))

income = participants * registration
expenses = food + decoration + other
balance = income - expenses

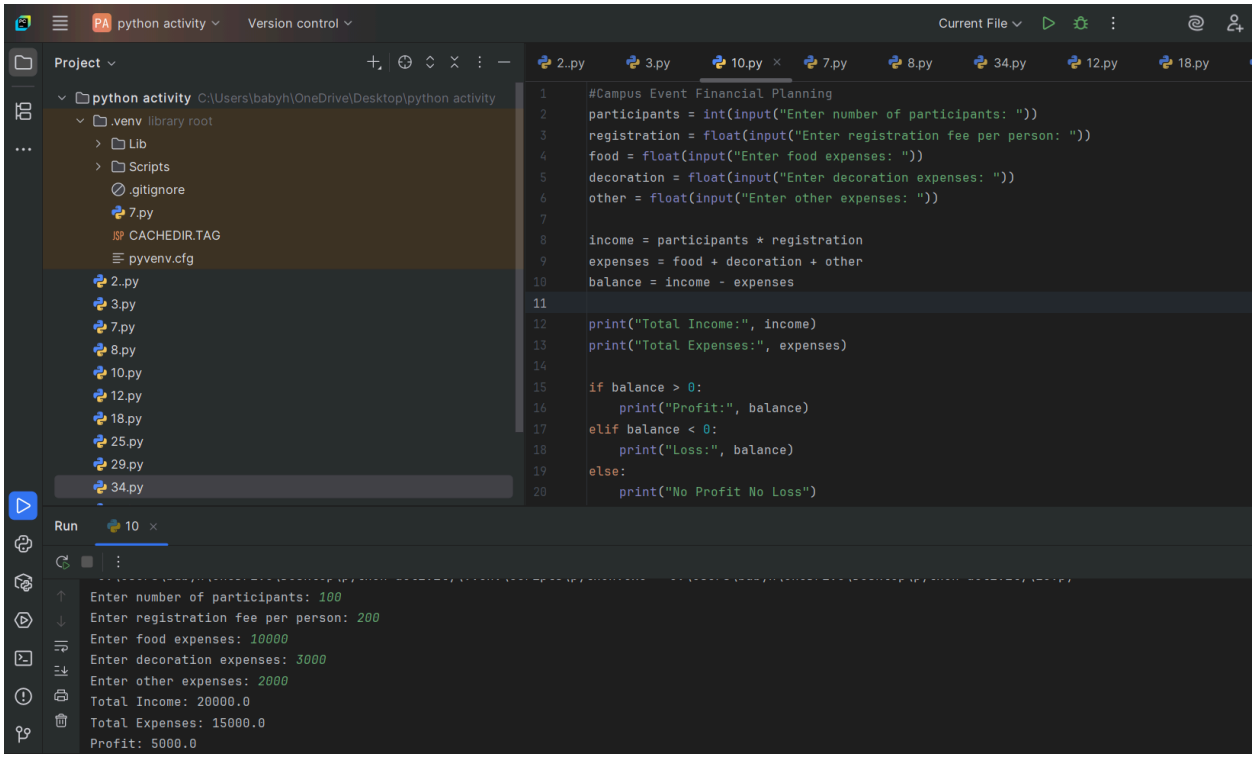
print("Total Income:", income)
print("Total Expenses:", expenses)
```

```
if balance > 0:
 print("Profit:", balance)
elif balance<0:
 print("Loss:", -balance)
else:
 print("No Profit No Loss")
```

8. Test Cases

| Participants | Fee | Expenses | Result |
|---|---|---|---|
| 100 | 200 | 15000 | Profit: 5000 |
| 50 | 100 | 6000 | Loss: 1000 |
| 20 | 100 | 2000 | No Profit No Loss |

9. Output Screenshot




10. Conclusion

The program calculates event income, expenses, and identifies whether the event results in profit or loss.

I'll follow this same medium-length format for the remaining case studies.

# Case Study 4: Retail Billing with Multiple Discount Rules

1. Problem Statement

Develop a Python program to calculate the final purchase amount after applying discounts based on the purchase value and customer membership.

2. Objectives

- Calculate discounts based on purchase amount.
- Apply an additional discount for members.
- Display the final bill amount.

3. Assumptions

- Purchases of ₹5000 or above receive a 10% discount.

- Purchases of ₹2000 or above receive a 5% discount.
- Members receive an additional 5% discount.

4. Algorithm

1. Start.
2. Input purchase amount and membership status.
3. Check the purchase amount and calculate the discount.
4. Check whether the customer is a member.
5. Apply an additional discount if applicable.
6. Calculate the final amount.
7. Display the bill details.
8. Stop.

5. Flowchart

Start → Input Amount and Membership → Calculate Discount → Check Membership → Calculate Final Amount → Display Bill → Stop.

6. Pseudocode

```
START
INPUT purchase amount
INPUT membership status

IF amount >= 5000
discount = 10% of amount
ELSE IF amount >= 2000
discount = 5% of amount
ELSE
discount = 0

IF member = yes
Add 5% discount

final amount = amount − discount

DISPLAY discount and final amount
STOP
```

7. Python Source Code

```
amount = float(input("Enter purchase amount: "))
member = input("Are you a member? yes/no: ")

if amount >= 5000:
 discount = amount * 10 / 100
elif amount >= 2000:
 discount = amount * 5 / 100
else:
 discount = 0

if member == "yes":
 discount = discount + amount * 5 / 100

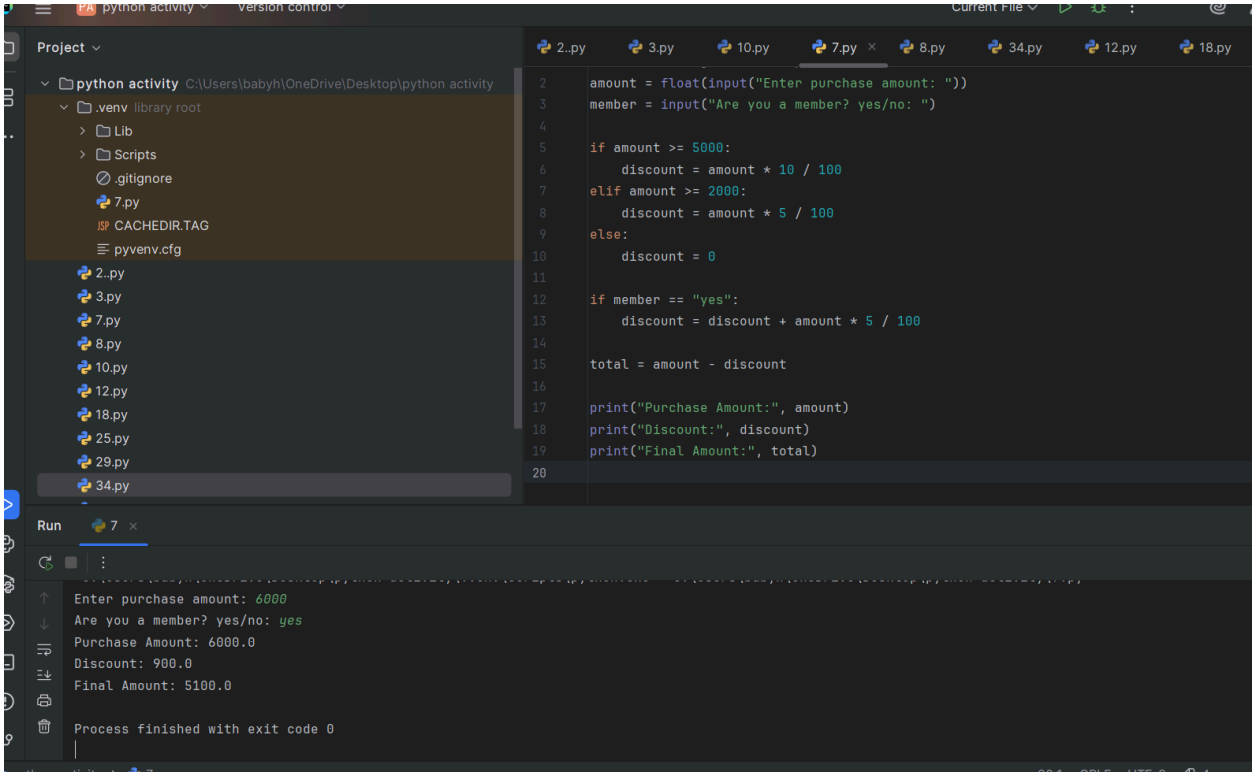
total = amount - discount

print("Purchase Amount:", amount)
print("Discount:", discount)
print("Final Amount:", total)
```

8. Test Cases

| Purchase Amount | Member | Discount | Final Amount |
|---|---|---|---|
| ₹6000 | yes | ₹900 | ₹5100 |
| ₹3000 | no | ₹150 | ₹2850 |
| ₹1000 | no | ₹0 | ₹1000 |

9. Output Screenshot




10. Conclusion

The program calculates the purchase discount based on the amount and membership status and displays the final bill.

# Case Study 5: Manufacturing Cost and Profitability Analysis

1. Problem Statement

Develop a Python program to calculate the total manufacturing cost, total income, and determine whether the company makes a profit or loss.

2. Objectives

- Calculate the total production cost.
- Calculate total income from sales.
- Find the profit or loss.

3. Assumptions

- Quantity, material cost, labour cost, and selling price are entered by the user.

- Material and labour costs are calculated per product.
- All products are sold.

4. Algorithm

1. Start.
2. Input quantity, material cost, labour cost, and selling price.
3. Calculate total cost.
4. Calculate total income.
5. Find profit or loss.
6. Display the results.
7. Stop.

5. Flowchart

Start → Input Details → Calculate Total Cost → Calculate Total Income → Find Profit/Loss → Display Result → Stop.

6. Pseudocode

START
INPUT quantity, material cost, labour cost, selling price

cost = (material cost + labour cost) × quantity
income = selling price × quantity
profit = income − cost

IF profit > 0, DISPLAY Profit
ELSE IF profit < 0, DISPLAY Loss
ELSE DISPLAY No Profit No Loss

STOP

7. Python Source Code

```
quantity = int(input("Enter quantity: "))
material = float(input("Enter material cost: "))
labour = float(input("Enter labour cost: "))
selling = float(input("Enter selling price: "))

cost = (material + labour) * quantity
income = selling * quantity
profit = income - cost
```

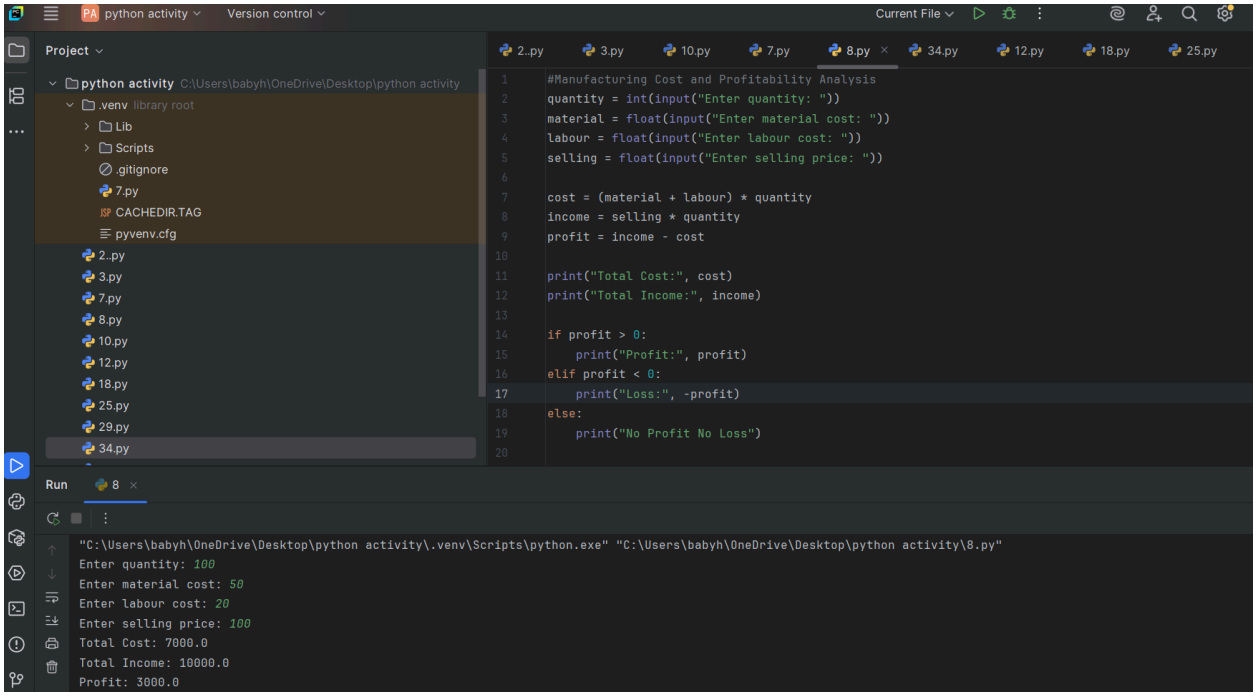
```
print("Total Cost:", cost)
print("Total Income:", income)

if profit > 0:
 print("Profit:", profit)
elif profit<0:
 print("Loss:", -profit)
else:
 print("No Profit No Loss")
```

8. Test Cases

| Quantity | Material | Labour | Selling Price | Result |
|---|---|---|---|---|
| 100 | 50 | 20 | 100 | Profit: ₹3000 |
| 50 | 40 | 30 | 60 | Loss: ₹500 |
| 20 | 30 | 20 | 50 | No Profit No Loss |

9. Output Screenshot




10. Conclusion

The program calculates manufacturing costs and sales income and identifies whether the company earns a profit or faces a loss.

# Case Study 6: Student Feedback Sentiment Scoring

1. Problem Statement

Develop a Python program to analyse student feedback by counting positive and negative words and identifying the overall sentiment.

2. Objectives

- Accept feedback from students.
- Count positive and negative words.
- Classify feedback as positive, negative, or neutral.

3. Assumptions

- Feedback is entered as a sentence.
- Words such as good, excellent, and nice are positive.
- Words such as bad, poor, and worst are negative.

4. Algorithm

1. Start.
2. Input student feedback.
3. Convert the feedback into lowercase words.
4. Count positive and negative words.
5. Compare both counts.
6. Display the sentiment.
7. Stop.

5. Flowchart

Start → Input Feedback → Count Positive and Negative Words → Compare Counts → Display Sentiment → Stop.

6. Pseudocode

START
INPUT feedback
SET positive = 0, negative = 0

FOR each word in feedback
IF word is positive
INCREASE positive
ELSE IF word is negative
INCREASE negative

IF positive > negative
DISPLAY Positive
ELSE IF negative > positive
DISPLAY Negative
ELSE
DISPLAY Neutral

STOP

7. Python Source Code

```
feedback = input("Enter student feedback: ")
positive = 0
negative = 0
```

```
for word in feedback.lower().split():
 if word == "good" or word == "excellent" or word == "nice":
 positive = positive + 1
 elif word == "bad" or word == "poor" or word == "worst":
 negative = negative + 1

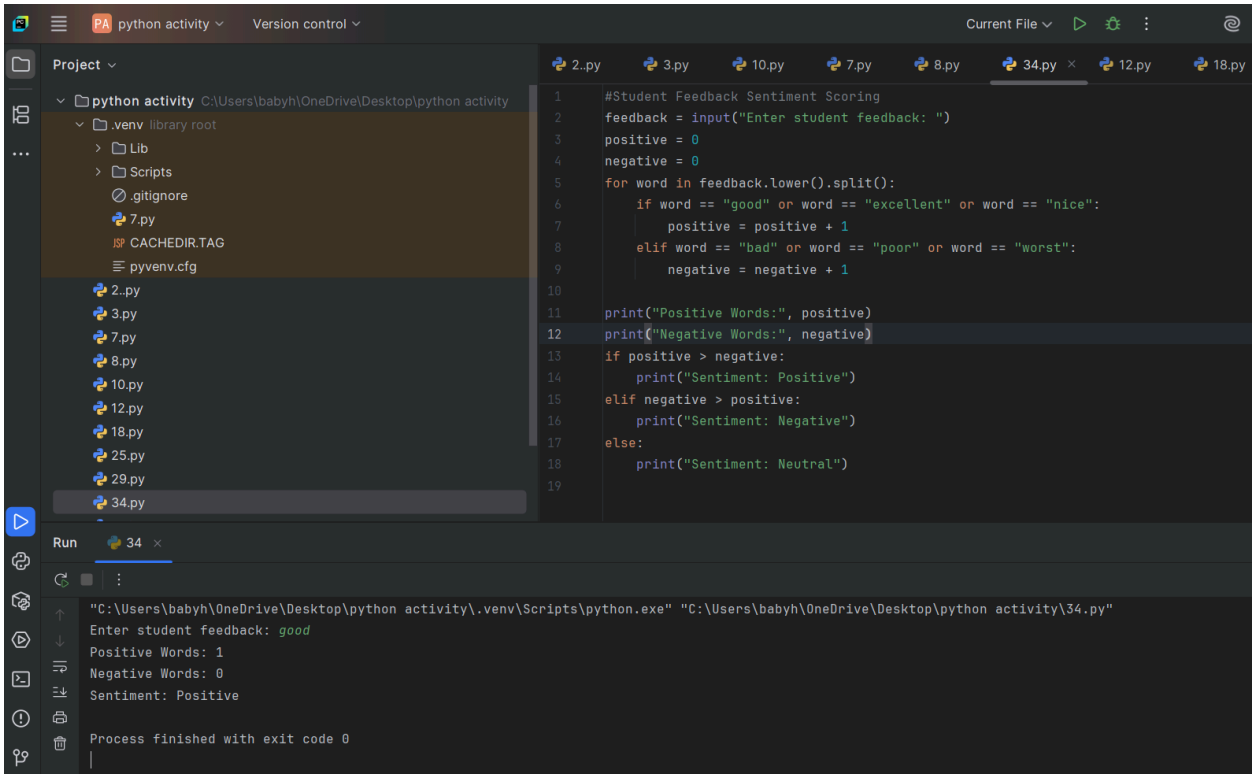
print("Positive Words:", positive)
print("Negative Words:", negative)

if positive > negative:
 print("Sentiment: Positive")
elif negative > positive:
 print("Sentiment: Negative")
else:
 print("Sentiment: Neutral")
```

8. Test Cases

| Feedback | Positive | Negative | Result |
|---|---|---|---|
| good excellent bad | 2 | 1 | Positive |
| bad poor nice | 1 | 2 | Negative |
| good bad | 1 | 1 | Neutral |

9. Output Screenshot




10. Conclusion

The program analyses student feedback by counting positive and negative words and displays the overall sentiment.

# Case Study 7: Attendance Recovery Computation

1. Problem Statement

Develop a Python program to calculate a student's current attendance percentage and find how many additional classes the student must attend to reach the required attendance percentage.

2. Objectives

- Calculate current attendance percentage.
- Check whether the required percentage is reached.
- Find the number of additional classes needed.

3. Assumptions

- Total classes and attended classes are entered by the user.

- The target attendance percentage is entered by the user.
- The student attends all upcoming classes.

4. Algorithm

1. Start.
2. Input total classes, attended classes, and target percentage.
3. Check whether the attendance details are valid.
4. Calculate current attendance percentage.
5. If the target is reached, display a message.
6. Otherwise, calculate the additional classes required.
7. Display the result.
8. Stop.

5. Flowchart

Start → Input Attendance Details → Validate Details → Calculate Current Percentage → Check Target → Calculate Additional Classes if Needed → Display Result → Stop.

6. Pseudocode

```
START
INPUT total classes, attended classes, target percentage

IF details are invalid
DISPLAY Invalid Details
ELSE
CALCULATE current percentage

   IF current percentage >= target
DISPLAY Requirement Satisfied
ELSE
SET required = 0
WHILE target is not reached
INCREASE required by 1
DISPLAY required classes
END IF

STOP
```

7. Python Source Code

```
total = int(input("Enter total classes: "))
attended = int(input("Enter attended classes: "))
target = int(input("Enter targeted attendance percentage: "))

if total total:
 print("Invalid attendance details")
else:
 current_percent = (attended / total) * 100
 print("Your current attendance is", current_percent)

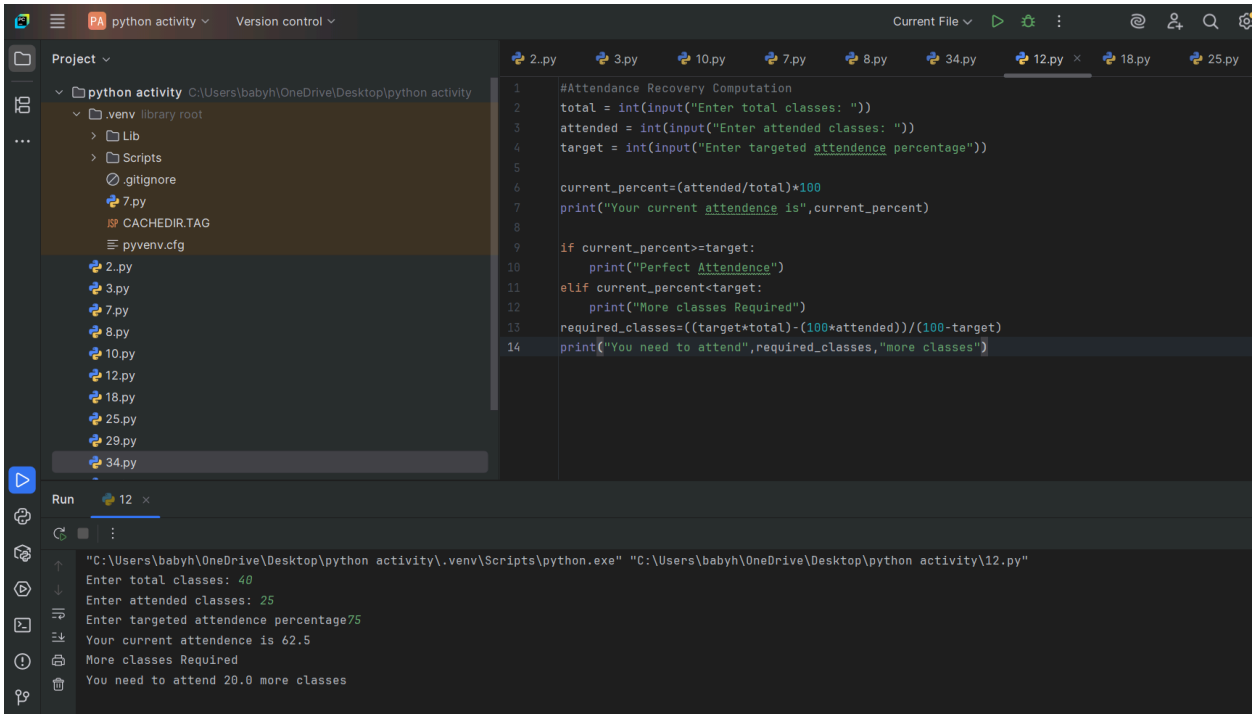
if current_percent >= target:
 print("Attendance Requirement Satisfied")
 else:
 required = 0
 while ((attended + required) / (total + required)) * 100<target:
 required = required + 1

print("You need to attend", required, "more classes")
```

8. Test Cases

| Total Classes | Attended | Target | Expected Result |
|---|---|---|---|
| 40 | 25 | 75% | 20 more classes |
| 40 | 35 | 75% | Requirement satisfied |
| 50 | 20 | 75% | 100 more classes |

9. Output Screenshot




10. Conclusion

The program calculates current attendance and determines how many additional classes a student must attend to reach the target percentage.

# Case Study 8: Fuel Consumption and Route Comparison

1. Problem Statement

Develop a Python program to compare the fuel expenses of two different routes. The program calculates the fuel required and total fuel cost for each route based on distance, vehicle mileage and fuel price. Finally, it displays which route is more economical.

2. Objectives

- To calculate fuel consumption for two routes.
- To calculate the total fuel cost.
- To compare the expenses of both routes.
- To identify the most economical route.

3. Assumptions

- Vehicle mileage remains constant.

- Fuel price is the same for both routes.
- Distance is measured in kilometres.
- Mileage is measured in kilometres per litre.

4. Algorithm

1. Start the program.
2. Enter the distances of Route 1 and Route 2.
3. Enter the vehicle mileage.
4. Enter the fuel price per litre.
5. Calculate fuel required for both routes.
6. Calculate the total fuel cost.
7. Compare the costs of both routes.
8. Display the economical route.
9. Stop the program.

5. Flowchart

[Insert the flowchart here.]

6. Pseudocode

START

INPUT distance1, distance2

INPUT mileage

INPUT fuel_price

fuel1 = distance1 / mileage

fuel2 = distance2 / mileage

cost1 = fuel1 * fuel_price

cost2 = fuel2 * fuel_price

DISPLAY cost1, cost2

IF cost1 < cost2

DISPLAY "Route 1 is economical"

ELSE IF cost2 < cost1

DISPLAY "Route 2 is economical"

ELSE

DISPLAY "Both routes have equal cost"

STOP

7. Python Source Code

```
distance1 = float(input("Enter distance of Route 1: "))

distance2 = float(input("Enter distance of Route 2: "))

mileage = float(input("Enter vehicle mileage: "))

fuel_price = float(input("Enter fuel price per litre: "))

fuel1 = distance1 / mileage

fuel2 = distance2 / mileage

cost1 = fuel1 * fuel_price

cost2 = fuel2 * fuel_price

print("Fuel required for Route 1:", fuel1, "litres")

print("Fuel required for Route 2:", fuel2, "litres")

print("Cost of Route 1: Rs.", cost1)

print("Cost of Route 2: Rs.", cost2)

if cost1 < cost2:

print("Route 1 is more economical")

elif cost2 < cost1:

print("Route 2 is more economical")

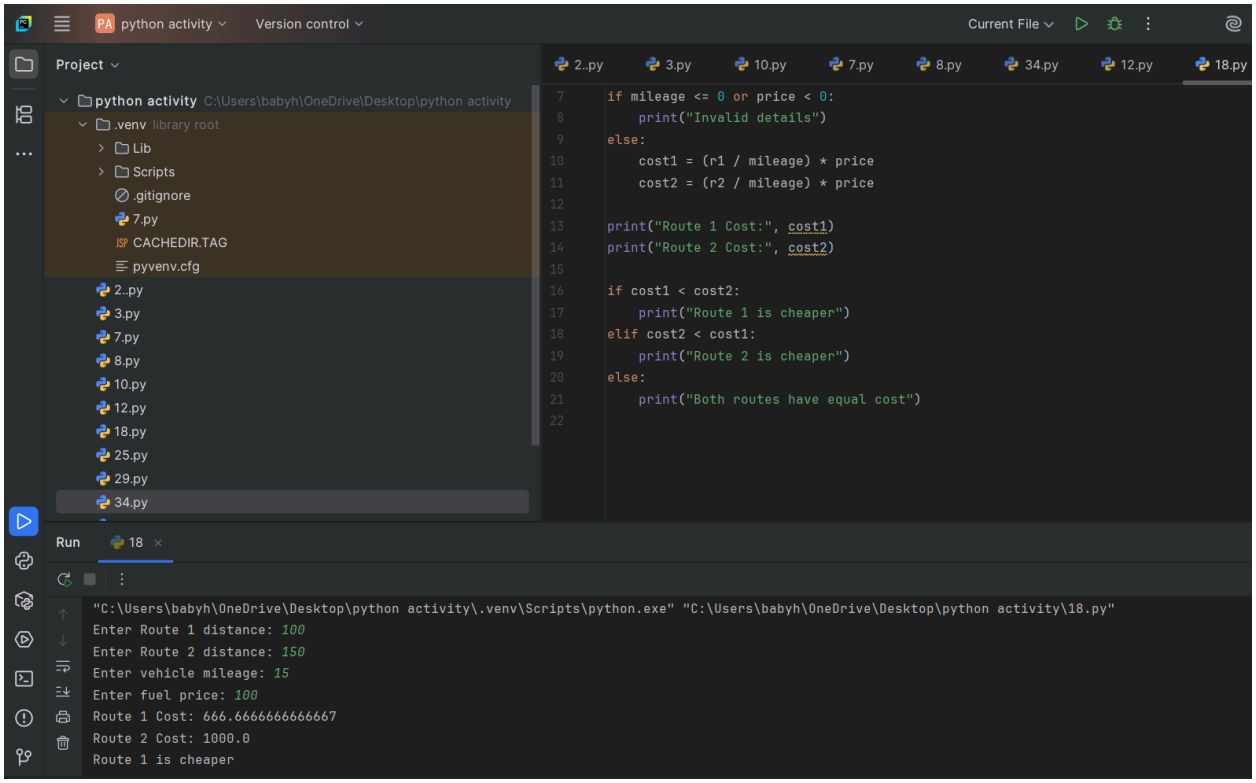
else:

print("Both routes have equal cost")
```

8. Test Cases

| Test case | Route 1 | Route 2 | Mileage | Fuel price |
|---|---|---|---|---|
| 1 | 100 km | 150 km | 20 km/l | ₹100 |
| 2 | 200 km | 100 km | 25 km/l | ₹100 |
| 3 | 120 km | 120 km | 20 km/l | ₹100 |

9. Output




10. Conclusion

The Python program successfully calculates fuel consumption and fuel expenses for two different routes. It compares the costs and identifies the route that requires less fuel expenditure. This helps users choose a more economical route.

# Case Study 9: Progressive Water Tariff Analysis

1. Problem Statement

Develop a Python program to calculate a water bill based on the number of units consumed using different tariff rates.

2. Objectives

- Calculate water charges based on usage.
- Apply different rates for different consumption levels.
- Display the total water bill.

3. Assumptions

- Water consumption is entered in units.
- First 100 units cost ₹2 per unit.
- Next 100 units cost ₹3 per unit.
- Above 200 units cost ₹5 per unit.

4. Algorithm

1. Start.
2. Input water units consumed.
3. Check whether the units are valid.
4. Calculate the bill using the applicable tariff.
5. Display the total bill.
6. Stop.

5. Flowchart

Start → Input Water Units → Check Validity → Calculate Bill According to Slabs → Display Bill → Stop.

6. Pseudocode

START
INPUT units

IF units < 0
DISPLAY Invalid Units
ELSE IF units = 0:
print("Water Units:", units)
print("Total Water Bill:", bill)

7.python source code

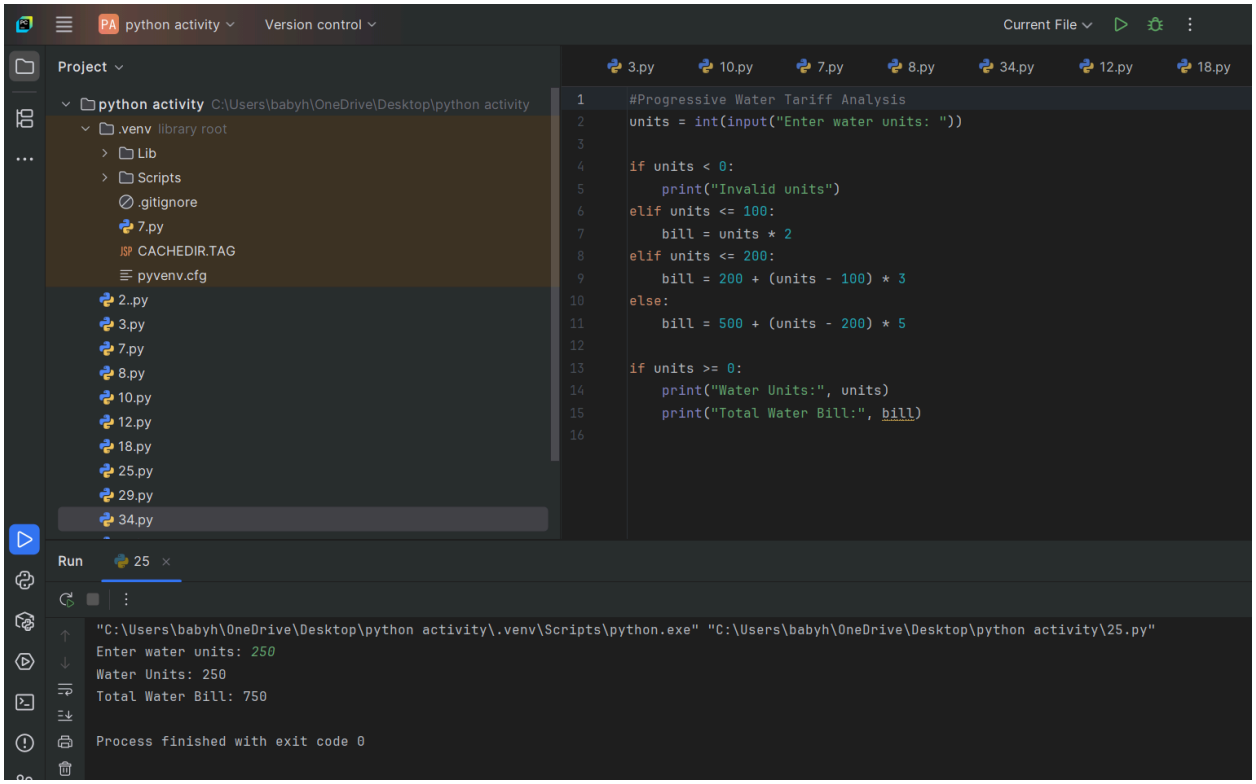
```
units = int(input("Enter water units: "))

if units <0:
 print("Invalid units")
elif units= 0:
 print("Water Units:", units)
 print("Total Water Bill:", bill)
```

8. Test Cases

| Units | Expected Bill |
|---|---|
| 80 | ₹160 |
| 150 | ₹350 |
| 250 | ₹750 |

9. Output Screenshot




10. Conclusion

The program calculates the water bill using progressive tariff rates based on water consumption.

# Case Study 10: Password Policy Validation Engine

## 1. Problem Statement

**Develop a Python program to check whether a password satisfies basic security requirements.**

## 2. Objectives

- **Check password length.**
- **Identify uppercase and lowercase letters.**
- **Check for digits and special characters.**
- **Display whether the password is valid.**

## 3. Assumptions

**A valid password must contain at least 8 characters, one uppercase letter, one lowercase letter, one digit, and one special character.**

**4. Algorithm**

1. **Start.**
2. **Input a password.**
3. **Initialize counters for different character types.**
4. **Check each character in the password.**
5. **Verify all password conditions.**
6. **Display whether the password is valid.**
7. **Stop.**

**5. Flowchart**

**Start → Input Password → Check Characters → Verify Password Conditions → Display Valid or Invalid → Stop.**

**6. Pseudocode**

```
START
INPUT password
SET upper, lower, digit, special = 0

FOR each character in password
CHECK its type
UPDATE the corresponding variable

IF length >= 8 AND all conditions are satisfied
DISPLAY Valid Password
ELSE
DISPLAY Invalid Password

STOP
```

**7. Python Source Code**

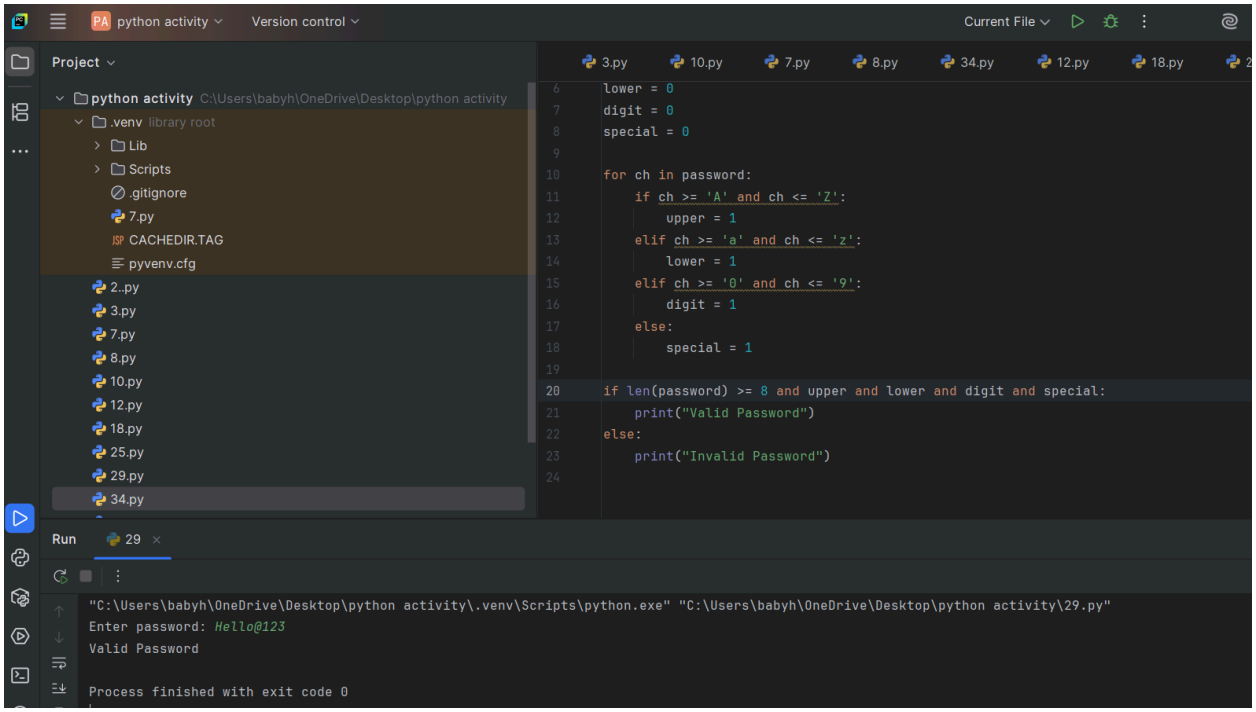
```
password = input("Enter password: ")
upper = 0
lower = 0
digit = 0
special = 0
```

**for ch in password:**
**if ch >= 'A' and ch = 'a' and ch = '0' and ch = 8 and upper and lower and digit and special:**
**print("Valid Password")**
**else:**
**print("Invalid Password")**

**8. Test Cases**

| Password | Expected Result |
|---|---|
| Hello@123 | Valid Password |
| hello123 | Invalid Password |
| HELLO@123 | Invalid Password |

**9. Output Screenshot**




## 10. Conclusion

**The program checks password requirements and identifies whether the entered password is valid.**